

第 1 章 编程语言与 Python 概述（第 1-4 页）

1.1 编程语言分类

编程语言是人和计算机沟通的媒介，按照执行方式可分为编译型语言与解释型语言两大类。编译型语言需要提前将完整代码编译为机器可识别的二进制文件，再运行程序，代表语言有 C、C++、Go 等。这类语言执行效率高，适合开发底层系统、嵌入式设备、高性能服务端程序。

解释型语言无需提前整体编译，代码在运行时由解释器逐行解析执行，开发流程更简洁，调试难度低。Python、JavaScript、PHP 都属于典型的解释型语言。随着技术发展，现代语言边界逐渐模糊，Python 也引入了即时编译（JIT）技术进一步提升运行速度。

按照应用场景划分，编程语言又可分为系统级语言、脚本语言、前端语言、数据分析语言等。Python 作为一门**全场景脚本语言**，覆盖了绝大多数开发领域，也是目前全球使用人数增长最快的编程语言之一。

1.2 Python 发展历史

Python 由荷兰程序员吉多·范罗苏姆（Guido van Rossum）于 1989 年圣诞节期间开始开发，1991 年正式发布第一个公开版本。语言命名灵感来自英国喜剧《蒙提·派森的飞行马戏团》，并非指代爬行动物蟒蛇，这也是很多初学者容易误解的知识点。

1994 年 Python 1.0 版本发布，支持类、异常处理、函数等核心面向对象特性；2000 年 Python 2.0 问世，加入列表推导式、垃圾回收机制，奠定了现代 Python 的语法基础。Python 2 系列持续维护至 2020 年，目前已全面停止官方支持。

2008 年 Python 3.0 正式发布，这是一次**不向下兼容**的重大版本迭代，重构了字符串编码、输入输出、类定义等核心逻辑，彻底解决了 Python 2 时代编码混乱、语法冗余的问题。目前主流使用版本为 Python 3.8 ~ Python 3.12，也是企业开发、学习的标准版本。

1.3 Python 核心优势

- 语法简洁优雅**：Python 采用强制缩进作为代码格式，摒弃了大括号、分号等冗余符号，伪代码风格的语法降低了入门门槛。相同功能的代码，Python 代码量通常仅为 C++、Java 的 1/3~1/2。
- 跨平台特性**：支持 Windows、macOS、Linux、国产操作系统、嵌入式设备等几乎所有主流平台，一份代码无需大幅修改即可跨系统运行。
- 海量第三方库**：Python 拥有全球最完善的开源库生态，内置标准库覆盖文件操作、网络请求、多线程等基础功能，第三方库可实现数据分析、人工智能、Web 开发、自动化运维、爬虫等所有场景。

4. **应用场景极广**：从简单的桌面小工具、办公自动化脚本，到大型分布式系统、大模型训练、云计算服务，Python 均有成熟解决方案。

1.4 Python 主流应用领域

1.4.1 数据分析与可视化

这是 Python 最经典的应用领域之一。依托 Pandas、NumPy、Matplotlib、Seaborn 等库，开发者可以快速完成数据清洗、统计分析、报表生成、可视化图表绘制。目前金融、电商、政务、传统制造业的数据分析岗位，Python 均为必备技能。

1.4.2 人工智能与大模型开发

深度学习框架 TensorFlow、PyTorch、PaddlePaddle 均以 Python 为主要开发语言。从传统机器学习、计算机视觉、语音识别，到当下热门的大语言模型、RAG 检索增强生成、智能体开发，Python 都是行业首选语言。

1.4.3 Web 后端开发

主流 Web 框架包括 Django（全能重型框架）、Flask（轻量微型框架）、FastAPI（高性能异步框架）。FastAPI 凭借自动接口文档、异步支持、高并发能力，成为当下 API 服务、后端接口开发的热门选择。

1.4.4 自动化与爬虫

利用 Python 可以编写自动化脚本，实现办公自动化（Excel/Word 批量处理）、运维自动化（服务器批量操作）、浏览器自动化等。配合 Requests、Scrapy 等爬虫库，可合规完成公开网络数据的采集工作。

1.4.5 运维与云计算

在 Linux 运维、容器化（Docker）、容器编排（K8s）、云平台二次开发场景中，Python 脚本是运维工程师日常工作的核心工具，灵活高效且易于维护。

1.5 学习 Python 的前置要求

学习 Python 无需深厚的计算机底层知识，零基础人群也可入门。基础要求仅为：掌握电脑基本操作、了解文件目录概念、具备基础逻辑思维。如果目标是深耕人工智能、后端开发等方向，则需要后续补充计算机网络、数据结构、算法等知识。

第 2 章 环境搭建与开发工具（第 5-9 页）

2.1 Windows 系统环境安装

2.1.1 官方解释器下载

访问 Python 官方网站（www.python.org），进入 Downloads 板块，选择对应 Windows 版本。建议优先选择**64 位安装包**，匹配主流电脑配置。下载时区分可执行安装包（exe）与压缩包，新手推荐 exe 安装包。

运行安装程序时，**必须勾选界面底部「Add Python to PATH」选项**，该选项会自动将 Python 路径加入系统环境变量，避免后续出现“python 不是内部或外部命令”的常见报错。

自定义安装路径建议选择非系统盘，例如 D:\Python311，不要使用中文、空格、特殊字符命名文件夹，防止编码和路径异常。

2.1.2 安装验证

安装完成后，按下 Win+R 打开运行窗口，输入 cmd 启动命令提示符。依次执行两条命令：

plaintext

```
python --version  
pip --version
```

若正常输出版本号，则代表解释器与包管理工具 pip 安装成功。若提示命令不存在，说明环境变量配置失败，需要手动添加 Python 安装目录下的 python.exe 路径与 Scripts 目录路径。

2.2 macOS 与 Linux 环境安装

macOS 系统自带 Python 2.7 版本，不建议直接使用系统默认 Python，避免破坏系统依赖。推荐使用 Homebrew 包管理器安装新版 Python，终端执行命令：

plaintext

```
brew install python3
```

主流 Linux 发行版（Ubuntu、CentOS）可通过系统自带包管理器安装：

- Ubuntu/Debian：`sudo apt install python3 python3-pip`
- CentOS/RHEL：`sudo yum install python3 python3-pip`

Linux 与 macOS 下默认使用 `python3`、`pip3` 作为命令，区分系统旧版 Python 2。

2.3 虚拟环境详解

2.3.1 虚拟环境作用

Python 全局环境存在版本冲突、库版本混乱的问题：不同项目可能依赖同一个库的不同版本，全局安装会导致项目无法运行。**虚拟环境**可以为每个项目创建独立的隔离运行环境，每个环境拥有独立的解释器、第三方库，互不干扰。

虚拟环境是企业开发的标准规范，正式项目必须使用虚拟环境。

2.3.2 内置 venv 模块使用

Python 3.3 及以上版本内置 `venv` 模块，无需额外安装。使用步骤：

1. 打开终端 / CMD，进入项目文件夹

2. 创建虚拟环境：`python -m venv venv`（第二个 `venv` 为环境文件夹名，可自定义）

3. 激活虚拟环境

- Windows CMD：`venv\Scripts\activate`
- Windows PowerShell：`.\venv\Scripts\Activate.ps1`
- macOS/Linux：`source venv/bin/activate`

4. 退出虚拟环境：终端输入 `deactivate`

激活成功后，终端前缀会显示 `(venv)` 标识，代表当前处于隔离环境中。

2.3.3 第三方虚拟环境工具

`virtualenv` 是老牌虚拟环境工具，功能比原生 `venv` 更丰富；`conda`（Anaconda/Miniconda）是数据科学领域主流工具，不仅可以管理 Python 环境，还能管理 C/C++ 依赖库，适合数据分析、AI 项目。

2.4 主流开发编辑器与 IDE

2.4.1 轻量编辑器：VS Code

Visual Studio Code（VS Code）是免费跨平台代码编辑器，体积小、启动快，安装 Python 插件后即可实现代码高亮、语法提示、运行调试、终端集成等功能，兼顾新手与资深开发者，是目前使用最广泛的工具。

必备插件：Python、Pylance、Code Runner、Chinese (Simplified)。

2.4.2 专业 IDE：PyCharm

PyCharm 是 Python 专属集成开发环境，分为社区版（免费开源）与专业版（付费）。内置虚拟环境管理、项目结构管理、一键运行、断点调试、代码重构等功能，开箱即用，适合大型项目开发。缺点是启动速度较慢，资源占用高于 VS Code。

2.4.3 Jupyter Notebook

交互式编程工具，以单元格形式运行代码，支持代码、文本、公式、图片混合编辑，**专为数据分析、算法调试、笔记整理设计**，是数据科学家、AI 工程师的标配工具。代码分段执行，便于逐段测试逻辑、展示结果。

2.5 包管理工具 pip

pip 是 Python 官方包管理工具，负责第三方库的安装、升级、卸载、检索。核心常用命令：

1. 安装库：`pip install 库名`
2. 指定版本安装：`pip install 库名==版本号`
3. 升级库：`pip install --upgrade 库名`
4. 卸载库：`pip uninstall 库名`
5. 查看已安装库：`pip list / pip freeze`

国内访问官方源速度较慢，可配置国内镜像源（阿里云、清华源、豆瓣源）加速下载，这是国内开发者的常规优化操作。

第 3 章 基础语法与数据类型（第 10-18 页）

3.1 变量与标识符

变量是内存中存储数据的容器，Python 定义变量**无需声明数据类型**，直接 `变量名 = 值` 即可创建，属于动态类型语言。

标识符（变量名、函数名、类名）命名规则：

1. 由字母、数字、下划线组成，不能以数字开头
2. 严格区分大小写
3. 不能使用 Python 关键字（if、else、for、while、def 等）
4. 建议使用下划线命名法（`user_name`、`user_age`），见名知意，禁止使用拼音、无意义单字符。

Python 关键字可通过 `keyword` 库查询，关键字无法作为变量名使用。

3.2 六大基础数据类型

Python 内置六大标准数据类型，分为可变类型与不可变类型：

3.2.1 数字类型 (Number)

包含整数 (int)、浮点数 (float)、复数 (complex)、布尔值 (bool)。布尔值本质是特殊整数，`True` 等价于 1，`False` 等价于 0。

整数支持十进制、二进制、八进制、十六进制表示；浮点数存在精度误差，金融计算场景不建议直接使用 `float`，需使用 `decimal` 模块。

3.2.2 字符串 (String)

由单引号、双引号、三引号包裹的文本内容，**不可变类型**。三引号支持多行字符串，常用于大段文本、文档注释。

字符串常用操作：拼接、切片、查找、替换、分割、大小写转换、去除首尾空格。字符串是 RAG 系统中最常处理的数据类型，文本分块、嵌入计算、召回逻辑都基于字符串操作实现。

3.2.3 列表 (List)

有序可变序列，使用 `[]` 定义，可存储任意数据类型，支持增删改查、切片、排序、遍历。列表是 Python 最常用的容器类型，常用于存储批量数据、检索结果集合。

3.2.4 元组 (Tuple)

有序不可变序列，使用 `()` 定义，元素一旦定义无法修改。安全性高于列表，常用于存储固定数据、函数多返回值。

3.2.5 字典 (Dict)

键值对 (key-value) 结构，使用 `{}` 定义，Python 3.7 + 版本字典默认保留插入顺序。key 必须是不可变类型 (字符串、数字、元组)，value 可以是任意类型。字典查询效率极高，底层基于哈希表实现，RAG 系统中常用来存储文档元数据、索引映射关系。

3.2.6 集合 (Set)

无序不重复元素集合，自动去重，常用于数据去重、交集、并集、差集运算。

3.3 运算符

分为算术运算符、比较运算符、赋值运算符、逻辑运算符、成员运算符、身份运算符。

- 逻辑运算符：`and`（与）、`or`（或）、`not`（非），用于多条件判断
- 成员运算符：`in` / `not in`，判断元素是否存在于序列中，文本检索、内容匹配高频使用

3.4 分支语句（条件判断）

`if...elif...else` 分支结构，根据条件执行不同代码逻辑。Python 依靠缩进划分代码块，缩进必须统一（4 个空格为行业标准）。

嵌套 if 语句可实现多层条件判断，但过多嵌套会降低代码可读性，工程开发中建议控制嵌套层数。

3.5 循环语句

3.5.1 while 循环

基于条件判断循环，条件为真则持续执行代码块，适合不确定循环次数的场景。需注意设置循环终止条件，避免死循环。

3.5.2 for 循环

遍历循环，可遍历字符串、列表、元组、字典、文件对象等所有可迭代对象，是 Python 使用频率最高的循环结构。搭配 `range()` 函数可实现固定次数循环。

循环控制关键字：`break`（终止整个循环）、`continue`（跳过当前单次循环）。

3.6 注释规范

注释用于解释代码逻辑，不会被解释器执行。分为单行注释（`#`）和多行注释（三引号）。规范的注释是项目可维护性的基础，正式开发中要求关键逻辑、函数、类必须添加注释。

第 4 章 函数、模块与文件操作（第 19-27 页）

4.1 函数基础

函数是封装一段可重复使用代码的代码块，使用 `def` 关键字定义。作用：简化代码、降低冗余、逻辑解耦、便于维护。

函数组成：函数名、参数、函数体、返回值（`return`）。分为无参函数、有参函数、默认参数函数、关键字参数函数。

4.1.1 局部变量与全局变量

定义在函数内部的变量为局部变量，仅函数内部生效；定义在函数外部的变量为全局变量，整个文件生效。函数内修改全局变量需要使用 `global` 关键字声明。

4.1.2 函数文档字符串

函数首行的三引号文本为**文档字符串**，用于描述函数功能、参数含义、返回值说明，IDE 可自动解析生成接口文档，是标准开发规范。

4.2 模块与包

模块指单个 `.py` 代码文件，包是包含多个模块的文件夹（内置 `__init__.py` 文件）。Python 通过 `import` 语句导入模块与包，实现代码拆分与复用。

标准库模块：Python 自带无需额外安装，如 `os`（系统操作）、`sys`（解释器操作）、`json`（JSON 数据解析）、`re`（正则表达式）。

第三方模块：需要通过 `pip` 安装，是 Python 生态的核心。

4.3 文件读写操作

文件操作是 RAG 系统的基础能力，用于读取本地 PDF、Word、TXT 文档，写入检索结果、日志等。

4.3.1 基础 `open` 函数

语法：`open(文件路径, 打开模式, 编码格式)`

常用打开模式：

- `r`：只读模式（默认）
- `w`：写入模式，清空原有内容
- `a`：追加模式，在文件末尾新增内容
- `r+`：读写模式

编码格式统一指定 `encoding="utf-8"`，解决中文乱码问题。

4.3.2 上下文管理器 `with`

`with` 语句是 Python 推荐的文件读写写法，代码执行完毕后**自动关闭文件**，无需手动调用 `close()` 方法，避免文件句柄泄漏，稳定性更强。

4.3.3 大文件读取

读取 GB 级别的超大文档时，不建议一次性读取全部内容，会造成内存溢出。推荐按行读取、分块读取，这也是 RAG 处理长文档的核心思路。

4.4 正则表达式 `re` 模块

正则表达式用于复杂文本匹配、提取、清洗，在文档解析、文本预处理、关键词召回场景中必不可少。
`re` 模块核心方法：`match`、`search`、`findall`、`sub`。

常用正则规则：匹配数字、中文、英文、特殊符号、换行符等，RAG 文本清洗（去除乱码、多余空格、特殊字符）高度依赖正则。

4.5 JSON 数据处理

JSON 是跨语言通用的数据格式，结构与 Python 字典、列表高度兼容。`json` 模块可实现 Python 对象与 JSON 字符串的相互转换，RAG 系统常用 JSON 存储文档切片、向量索引、召回结果。

第 5 章 面向对象编程（第 28-34 页）

5.1 面向对象基本概念

Python 是纯面向对象语言，一切皆对象。面向对象三大特性：**封装、继承、多态**。相比面向过程编程，面向对象更适合开发大型、复杂、可扩展的项目，企业级 RAG 系统、大模型应用均采用面向对象架构设计。

类（Class）是对象的模板，对象（Instance）是类的实例。类定义属性（数据）和方法（行为）。

5.2 类与实例的创建

使用 `class` 关键字定义类，`__init__` 为构造方法，创建实例时自动执行，用于初始化实例属性。

实例属性属于单个对象，类属性属于整个类，所有实例共享。合理区分两者可以优化内存占用。

5.3 封装

将属性和方法封装在类内部，对外隐藏内部实现细节，仅暴露必要的调用接口。Python 通过命名规范实现私有属性 / 方法（以双下划线开头），提升代码安全性。

5.4 继承

一个类（子类）可以继承另一个类（父类）的属性和方法，实现代码复用。支持单继承与多继承，继承是框架开发、组件封装的核心思想。

5.5 多态

不同子类重写父类同名方法，调用同一方法时呈现不同执行效果，提升代码扩展性。

5.6 常用内置方法

`__str__`、`__len__`、`__getitem__` 等魔法方法，可以自定义对象的打印、长度获取、下标访问逻辑，让自定义类像列表、字典一样便捷使用。

第 6 章 综合案例：简易文档检索工具（第 35-42 页）

6.1 项目需求

实现一个极简版本地文档检索工具，功能包括：读取本地 TXT 文档、文本分块、关键词匹配召回、输出相关片段。该案例模拟**轻量化 RAG 核心流程**，可直接用于检索功能测试。

6.2 整体流程

1. 加载本地文档 → 2. 文本预处理（去空格、去换行） → 3. 文档分块 → 4. 接收用户查询词 → 5. 关键词匹配召回片段 → 6. 输出检索结果

6.3 代码实现与逐行解析

(完整可运行代码、参数说明、异常处理、优化方案)

6.4 功能测试与问题优化

测试不同长度查询词、模糊关键词、长文本片段的召回效果；分析分块大小对召回精度的影响、关键词匹配的局限性，引出向量检索、语义召回的必要性。

6.5 扩展方向

基于本案例扩展为完整 RAG 系统：接入向量化模型、构建向量索引、实现语义召回、添加重排逻辑、对接大模型生成答案。
